

Tutorial 3

1. What is the probability for the 3rd insertion to have exactly two collisions while using linear probing in the hash table?

Solution: Let the size of the hash table be $n \geq 3$. We assume that insertions take place via linear probing. Further, we assume that no deletions or insertions occur in the same sequence. Lastly, we assume that the hash function is uniform, i.e., for a random key, the likelihood of any hash value occurring is exactly $1/n$. After collisions, let the first two insertions occur at indices i and j .

If $((i+1) \bmod n) \neq j$ and $((j+1) \bmod n) \neq i$, then the third insertion cannot have two collisions. The reason is that if the hash value of the third key collides with the first key, then index $(i+1) \bmod n$ is vacant, and there is only one collision. If the hash value is at index j , then index $(j+1) \bmod n$ is vacant and hence, only one collision. Any other hash value implies no collision. Let us neglect this case.

We focus only on cases where i and j are adjacent to each other on the table. Let us re-label the indices as a and b such that $((a+1) \bmod n) = b$. In this case, for any constant $0 \leq c < n$, the absolute value of $P(a = c)$ (not conditional) can be thought of as the probability that $((i = c \text{ and } j = (i+1) \bmod n) \text{ or } (j = c \text{ and } i = (j+1) \bmod n))$. The first event occurs when the second hash value is either i or $(i+1) \bmod n$, with a chance of $(1/n) \cdot (2/n)$. The second event occurs when the second hash value is $(n+i-1) \bmod n$, with a chance of $(1/n) \cdot (1/n)$.

From the above, we can say that $P(a = c) = (3/n^2)$ for all constants c , we safely say that two collisions occur if and only if the third hash-value is a , which occurs with a probability of $\frac{1}{n}$. However, c and, therefore, a can take any value. Thus, the product $(3/n^2) \cdot (1/n)$ has to be summed up across all values (n times), giving us the required answer: $(3/n^2)$.

2. Given that k elements have to be stored using a hash function with target space n . What is the probability of the hash function having an inherent collision? What is an estimate of the probability of a collision in the insertion of n elements?

Solution: To compute the probability, we first count the negative case: no collisions at all. This means that the k keys get mapped to distinct k keys out of n . We assume that for a random key and for any constant c , $P(\text{hash}(k) = c) = 1/n$.

The first key has n choices of hash values. Given each choice of the first key, the second key has $n-1$ possible hash values such that there is no collision. Given the first two keys and hash values, the third has $n-2$ choices, and so on, till the k^{th} key has $n-k+1$ choices. Overall, the ordered k -tuple has $\prod_{i=0}^{k-1} (n-i)$ choices for no collision. All those are mutually exclusive events with likelihoods $(1/n)^k$ each, being the product of k probabilities of $1/n$ for k independent events and their respective outcomes. Thus,

$$P(\text{No collisions}) = \frac{n!}{(n-k)! \cdot n^k}$$

One minus this expression gives the likelihood of a collision.

If $k = n$, the expression boils down to $(n!/n^n)$. We apply Stirling's approximation here:

$$\sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n+1}} < n! < \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\frac{1}{12n}}$$

$$\sqrt{2\pi n} \left(\frac{1}{e}\right)^n e^{\frac{1}{12n+1}} < \frac{n!}{n^n} < \sqrt{2\pi n} \left(\frac{1}{e}\right)^n e^{\frac{1}{12n}}$$

For large n , we can neglect the terms $e^{(1/(12n+1))}$ and $e^{(1/12n)}$ as both are very close to 1. However, the term $\sqrt{2\pi n} \cdot e^{-n}$ tends to zero as n grows very large. This means that the required probability is sandwiched between terms that exponentially decay toward zero.

The implication of the above is the likelihood that all n keys get uniquely mapped to the n possible hash values decay exponentially with n , and thus, collisions are almost certain for large n . This can be linked with the birthday paradox in that in a class of 365; it is almost impossible for all students to have different birthdays; even a single tutorial batch has a high likelihood of at least two birthdays being the same. In the context of hashing, although the expected number of terms sharing a hash value is one per hash value, we expect a good number of collisions to occur during hashing.

3. Let $C(i)$ be the chain of array indices that are queried to look for a key k in linear probing where $h(k) = i$.
 - (a) How does this chain extend by an insertion, and how does it change by a deletion?
 - (b) A search for a key k ends when an empty cell is encountered. What if we mark the end of $C(i)$ with an end marker? We stop the search when this marker is encountered. Would this work? Would this be efficient?
 - (c) Is there a way of not using tombstones?

Solution: To recall, linear probing covers indices $i, i + 1, \dots$ until an empty index is found (then k not found in the table) or the cell stores k . Note that tombstones do not count as empty cells in the hash table; counting them so after deletion would break existing chains and result in false negatives when searching. Including them means no change in these chains, answering part of the first (a) subproblem.

If the chain terminates at k (key found), then inserting a new element does not change anything. If the added element is k itself, then it occupies the position at the end of the chain, a vacant cell. So, no change. In the other case, the chain extends by one if the inserted element is at the end of the chain and is not k . The chain could extend a lot more if the inserted element joins the existing chain with another series of filled cells in the table. Note that the chain, except for the end, has only filled or tombstone cells; thus, the insertion cannot occur in the middle of the chain. If the insertion of the element that is not k does not occur at the end of the previous chain for searching k , then there is no change.

The answer to the second (b) part is that it would work but not be any more efficient than the standard algorithm. For this to work, an empty cell once encountered during the search should be marked an 'end.' Otherwise, the first search would go on an infinite loop, looking for an end marker. If this is the case, the search must look for both end marks and empty cells in each iteration, making it slightly less efficient but at the same time complexity.

For the third (c) part, a slight modification should be made to the algorithm discussed above: all cells should be marked 'end' at the start. Otherwise, a deletion without a tombstone would cause false negatives in future searches, just like the discussions without tombstones. If this modification is made, then we basically replace "empty" with an "end" marker and "tombstone" marker with "empty". The algorithm is, otherwise, identical to the normal algorithm with tombstones, and hence, we really do

not avoid tombstones, although we do not use them. Note that marking the freed cells as 'end' on deletions would be as bad as not using tombstones, which results in false negatives.

4. Let $m = 11$, $h_1(k) = (k \bmod 11)$, $h_2(k) = 6 - (k \bmod 6)$.
Let us use the following hash function for an open addressing scheme.

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

1. What will be the state of the table after insertions of 41, 22, 44, 59, 32, 31, and 74?
2. Let $h_2(k) = p - (k \bmod p)$. What should be the relationship between p and m such that h is a valid function for linear probing?
3. What is the average number of probes for an unsuccessful search if the table has α load factor?
4. What is the average time for a successful search?

Solution: To recall, the standard linear probing uses the function $h(k, i) = (h(k) + i) \bmod m$ where m is the size of the hash table [Here 11]. The way it works is that we check the cells indexed at $h(k, 0), h(k, 1), h(k, 2), \dots$ till we find an empty cell $h(k, a)$ for some a . The same logic should be extended here, except $h(k, i)$, which is defined in the problem statement.

Let the table before adding 41 be

--	--	--	--	--	--	--	--	--	--	--

Now, $h_1(41) = 8$ and $h_2(41) = 1$. This means that $h(41, 0) = 8$ and cell 8 is empty. So, we add 41 there:

								41		
--	--	--	--	--	--	--	--	----	--	--

$h_1(22) = 0$ and cell 0 is empty so we can add 22 there. [$h(k, 0) = h_1(k)$]

22								41		
----	--	--	--	--	--	--	--	----	--	--

$h_1(44) = 0$ but cell 0 is filled. Now, $h_2(44) = 4$ and $h(44, 1) = 4$ which is an empty tile.

22				44				41		
----	--	--	--	----	--	--	--	----	--	--

$h_1(59) = 4$ which is a filled cell. $h_2(59) = 1$ and $h(59, 1) = 5$ is an empty tile.

22				44	59			41		
----	--	--	--	----	----	--	--	----	--	--

$h_1(32) = 10$ and $h_1(31) = 9$ are both empty and different cells, so we can add them.

22				44	59			41	31	32
----	--	--	--	----	----	--	--	----	----	----

Lastly, $h_1(74) = 8$, which is occupied. $h_2(74) = 4$. Now, $h(74, 1) = 1$ which is free.

22	74			44	59			41	31	32
----	----	--	--	----	----	--	--	----	----	----

This is how the table looks after inserting the elements in the given order.

Moving on to the next part, a valid function for linear probing should eventually cover all cells in the grid as i increments. This is not possible if $\gcd(h_2(k), m) > 1$, for one or more values of k . The reason is that $h(k, i) \bmod \gcd(h_2(k), m)$ will be fixed, and hence, $h(k, i) \bmod m$ will only take some values. But if that gcd is 1, then it means that for all $0 \leq i < m$, $\gcd \cdot i \bmod m$ will have a unique value, and the number of these unique values will be m , so all cells of the table will be covered.

Note that $\gcd(h_2(k), m) = 1$ should hold for all values of k , not just for any one k . Keeping $h_2(k) = p - (k \bmod p)$ can take any value in $1, 2, \dots, p$ depending on the value of k . Thus, none of the first p natural numbers should share a common factor with m . In other words, the smallest prime factor (not 1) of m , the size of the hash table, should be larger than p . It is not necessary that m is prime – for $p = 6$, $m = 7 \cdot 13 = 91$ also works.

Exact expression of expected number of probes in a search (unsuccessful/successful) is mathematically complex hence we'll prove an appropriate upper bound for them. Claim: The mentioned expression for an unsuccessful one is atmost $1/(1 - \alpha)$. This is assuming $\alpha < 1$.

Proof: Let X be the number of probes made in an unsuccessful search, and let A_i be the event that an i^{th} probe occurs and it is to an occupied slot. Then $X \geq i$ when the event $A_1 \cap A_2 \cap \dots \cap A_{i-1}$ occurs. Hence,

$$P(X \geq i) = P(A_1 \cap A_2 \cap \dots \cap A_{i-1}) \\ = P(A_1) \times P(A_2|A_1) \times P(A_3|A_2 \cap A_1) \times \dots \times P(A_{i-1}|A_1 \cap \dots \cap A_{i-2})$$

$P(A_1) = n/m$ because there are n elements and m slots. For $j > 1$, the probability that there is a j^{th} probe and it is to an occupied slot, given that the first $j - 1$ probes were to occupied slots, is $(n - j + 1)/(m - j + 1)$ because we are finding one of the remaining $(n - (j - 1))$ elements in one of the $(m - (j - 1))$ unexamined slots, and we have uniform hashing so each of the slots is equally likely to be chosen next.

Because we have $n < m$, we have $(n - j)/(m - j) \leq n/m \forall 0 \leq j < m$. Thus,

$$P(X \geq i) = \frac{n}{m} \times \frac{n-1}{m-1} \times \dots \times \frac{n-i+2}{m-i+2} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$$

So $E[X] = \sum_{i=1}^{m-1} P(i^{th} \text{ probe occurs}) = \sum_{i=1}^{m-1} P(X \geq i) \leq \sum_{i=1}^{m-1} \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = 1/(1 - \alpha)$. Hence proved.

Claim: Expected number of probes for a successful search is atmost $\frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right)$.

Proof: A search for a key k reproduces the same probe sequence as when the element with key k was inserted. If k was the $(i + 1)^{th}$ key inserted into the hash table, there were only i keys in the table at the time, so the expected number of probes made in a search for k is at most $1/(1 - i/m) = m/(m - i)$ (as an insertion is equivalent to an unsuccessful search, here $\alpha = i/m$). Averaging over all n keys in the hash table, we get,

$$\begin{aligned} \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} &= \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} \\ &= \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \\ &\leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx \\ &= \frac{1}{\alpha} \ln\left(\frac{m}{m-n}\right) \\ &= \frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) \end{aligned}$$

Hence Proved.

5. Suppose you want to store a large set of key-value pairs, for example, $(name, address)$. You have operations in this set: addition, deletion, and search of elements. You also have queries about whether a particular name or address is in the set, and if so, then count them and delete all such entries. How would you design your hash tables?

Solution: Let us recall that in the classic hashing, the hash function operates on the key, and the value is simply stored alongside the key. An intuitive idea is to keep the key (from the point of view of hashing) as **both** the name and the address, with no special value otherwise. The hash function operates on this and gives us the index. Unfortunately, this gives rise to a problem: there are multiple different hash values for a given name, and hence, searching for the existence of an entry given a name (but not address – we do not know which addresses are there given this name) is as bad as linearly iterating through the table.

One way to get around this is to use a two-level hash table: the outer table N stores the keys as the names alone, and each value corresponding to a name points to a hash table itself. This inner hash table $N[name]$ stores only addresses and only addresses where the name is the corresponding key of the outer table. This makes finding elements based on names easy but does not solve the problem of searching for given addresses.

To solve that, we can use a double two-level hash table: one two-level table N as described above and one two-level table A done the other way round, i.e., A stores the keys as addresses and values as pointers to inner hash tables $A[address]$ that store names corresponding to addresses. Note that this requires additional storage space (double) and more maintenance.

Specifically, it is important to ensure integrity: for every $(name, address)$ pair in the set, $N[name]$ should exist and contain $address$, and $A[address]$ should exist and contain $name$. Insertion should be done in both tables at once. On the other hand, if any $(name, address)$ pair does not exist, then $A[address]$, if it exists, should not contain $name$, and $N[name]$, if it exists, should not contain $address$. This should be reflected in the deletion of pairs.